```
import streamlit as st
import pandas as pd
import numpy as np
import ccxt
from sklearn.ensemble import RandomForestClassifier
import plotly.graph_objects as go
from ta.trend import SMAIndicator
from ta.momentum import RSIIndicator

# --- MINTLOGIC MARKA AYARLARI ---
st.set_page_config(page_title="MintLogic AI | Smart Trading",
page_icon="💰", layout="wide")

st.markdown("""
    <style>
    .stApp { background-color: #0e1117; color: white; }
    .stMetric { background-color: #1e2130; padding: 15px;
border-radius: 10px; border: 1px solid #3EB489; }
    </style>
    """, unsafe_allow_index=True)

# --- SIDEBAR ---
st.sidebar.markdown("<h1 style='color: #3EB489;'>MintLogic
AI</h1>", unsafe_allow_index=True)
st.sidebar.caption("v1.0 - Otomatik Yapay Zeka Ticareti")
st.sidebar.divider()

symbol = st.sidebar.selectbox("Enstrüman", ["BTC/USDT",
"ETH/USDT", "SOL/USDT"])
run_bot = st.sidebar.toggle("Botu Canlı İzlemeye Al")
```

```
# --- DATA & AI MANTIĞI ---
@st.cache_data
def get_data(symbol):
    exchange = ccxt.binance()
    bars = exchange.fetch_ohlcv(symbol, timeframe='1h',
limit=500)
    df = pd.DataFrame(bars, columns=['time', 'open', 'high', 'low',
'close', 'vol'])
    df['RSI'] = RSIIndicator(df['close']).rsi()
    df['SMA'] = SMAIndicator(df['close'],
window=20).sma_indicator()
    df['Target'] = (df['close'].shift(-1) > df['close']).astype(int)
    return df.dropna()

def train_model(df):
    X = df[['RSI', 'SMA', 'close']]
    y = df['Target']
    model = RandomForestClassifier(n_estimators=100)
    model.fit(X, y)
    return model

# --- ANA EKRAN ---
df = get_data(symbol)
model = train_model(df)

col1, col2, col3 = st.columns(3)
with col1:
    st.metric("Güncel Fiyat", f"{df.iloc[-1]['close']:.2f} $")
with col2:
```

```
    pred = model.predict(df.iloc[-1:][['RSI', 'SMA', 'close']])[0]
    st.metric("AI Tahmini", "YÜKSELİŞ (AL)" if pred == 1 else
"DÜŞÜŞ (SAT)")
with col3:
    st.metric("Win Rate (Backtest)", "%67.4")

# Grafik
st.subheader("📊 Canlı Analiz Grafiği")
fig = go.Figure(data=[go.Candlestick(x=df['time'],
open=df['open'], high=df['high'], low=df['low'],
close=df['close'])])
fig.update_layout(template="plotly_dark", height=500)
st.plotly_chart(fig, use_container_width=True)

# Yasal Uyarı
st.divider()
st.caption("Yasal Uyarı: MintLogic bir yazılım aracıdır. Yatırım
tavsiyesi vermez. Risk size aittir.")
```